

740. Delete and Earn

```
class Solution {
    int[] dp;
    int[] points;

    public int deleteAndEarn(int[] nums) {

        points = new int[10001];
        for (int num : nums) points[num] += num;
        dp = new int[10001];
        for (int i = 0; i < dp.length; i++) dp[i] = -1;

        return solve(10000);
    }

    int solve(int i) {
        if (i <= 0) return 0;
        if (dp[i] != -1) return dp[i];
        return dp[i] = Math.max(solve(i - 1), points[i] + solve(i - 2));
    }
}
```

416. partition equal subset sum

```
class Solution {
    Boolean [][] dp;
    public boolean solve(int n,int[] nums,int target){
        if(target==0) return true;
        if(n==0) return nums[0]==target;
        if(dp[n][target]!=null){
            return dp[n][target];
        }
        boolean notpick=solve(n-1,nums,target);
        boolean pick=false;
        if(nums[n]<target)
            pick=solve(n-1,nums,target-nums[n]);
        return dp[n][target]=pick||notpick;
    }
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int num : nums)
            sum += num;
        if (sum % 2 != 0)
            return false;
    }
}
```

```

        dp=new Boolean[nums.length][sum/2+1];
        return solve(nums.length - 1, nums, sum / 2);
    }
}

```

494. target sum

```

class Solution {
    int[][] dp;
    int solve(int idx,int target,int[] nums){
        if(idx==0){
            if(target==0&&nums[0]==0) return 2;
            if(target==0||target==nums[0])return 1;
            return 0;
        }
        if(dp[idx][target]!=-1)
            return dp[idx][target];
        int notpick=solve(idx-1,target,nums);
        int pick=0;
        if(nums[idx]<=target){
            pick=solve(idx-1,target-nums[idx],nums);
        }
        return dp[idx][target]=pick+notpick;
    }

    public int findTargetSumWays(int[] nums, int target) {
        int sum=0;
        for(int x:nums) sum+=x;
        if(Math.abs(target)>sum) return 0;
        if((sum+target)%2!=0) return 0;
        int req=(sum+target)/2;
        dp=new int[nums.length][req+1];
        for(int[] row:dp)
            Arrays.fill(row,-1);
        return solve(nums.length-1,req,nums);
    }
}

```

983. Minimum Cost For Tickets

```
class Solution {
    int[] dp;

    public int mincostTickets(int[] days, int[] costs) {
        dp = new int[days.length];

        for (int i = 0; i < dp.length; i++) {
            dp[i] = -1;
        }

        return solve(0, days, costs);
    }

    int solve(int i, int[] days, int[] costs) {
        if (i >= days.length) return 0;

        if (dp[i] != -1) return dp[i];
        int oneDay = costs[0] + solve(i + 1, days, costs);
        int j = i;

        while (j < days.length && days[j] < days[i] + 7) j++;

        int sevenDay = costs[1] + solve(j, days, costs);
        j = i;
        while (j < days.length && days[j] < days[i] + 30) j++;
        int thirtyDay = costs[2] + solve(j, days, costs);

        dp[i] = Math.min(oneDay, Math.min(sevenDay, thirtyDay));
        return dp[i];
    }
}
```

1049. Last Stone Weight II

```
class Solution {
    public int lastStoneWeightII(int[] stones) {

        int sum = 0;

        for (int i : stones) sum += i;

        int target = sum/2;
```

```
int[] dp = new int[target + 1];
for (int i : stones)
    for (int j = target; j >= i; j--)
        dp[j] = Math.max( dp[j], dp[j - i] + i );

return sum - 2 * dp[target];
}
```